# zs ai platform 接口文档

# HTTP API

ZS AI平台的RESTful API的完整参考。在继续之前，请确保您准备好您的ZS AI平台API key以进行身份验证。

## 知识库管理

### 创建

**POST** `/api/v1/datasets`

创建知识库。

#### Request

- Method: POST
- URL: `/api/v1/datasets`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"name"： string`

- "avatar": string
- "description": string
- "language": string
- "embedding_model": string
- "permission": string
- "chunk_method": string
- "parser_config": object

#### Request example

```
curl --request POST \
     --url http://{address}/api/v1/datasets \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '{
      "name": "test_1"
      }'
```

#### Request parameters

- "name": (*Body parameter*), `string`, *Required*
  要创建的知识库的唯一名称。必须遵守以下要求：
  - 允许的字符包括：
  - 英文字母（a-z，a-z）
  - 数字（0-9）
  - “_”（下划线）
  - 必须以英文字母或下划线开头。
  - 最多65535个字符。 不区分大小写

- `"avatar"`：(*Body parameter*)，`string`
  Base64编码。

- `"description"`：(*Body parameter*)，`string`
  要创建的知识库的简要说明。

- `"language"`：(*Body parameter*)，`string`
  要创建的知识库的语言设置。可用选项：

  - `"English"`（default）
  - `"Chinese"`

- `"embedding_model"`：(*Body parameter*)，`string`
  要使用的嵌入模型的名称。例如：“BAAI/bge-zh-v1.5”`

- `"permission"`：(*Body parameter*)，`string`
  指定谁可以访问要创建的知识库。可用选项：

  - `"me"`：（默认）只有您可以管理知识库。
  - `"team"`：所有团队成员都可以管理知识库。

- `"chunk_method"`：(*Body parameter*)，`enum<string>`
  要创建的知识库的分块方法。可用选项：

  - `"naive"`：通用（默认）
  - `"manual"`：手册
  - `"qa"`：Q&A
  - `"table"`：表格
  - `"paper"`：论文
  - `"book"`：书籍
  - `"laws"`：法律

    - `"presentation"`：演示文稿
    - `"picture"`：图片
    - `"one"`：不做切分的文档

- `"parser_config"`：(*Body parameter*)，`object`
  知识库解析器的配置设置。此JSON对象中的属性因所选的 `"chunk_method"` 而异：
    - 如果 `"chunk_method"` 是 `"naive"`，`"parser_config"` 对象包含如下属性:
        - `"chunk_token_count"`：默认 `128` 。
        - `"layout_recognize"`：默认 `true` 。
        - `"html4excel"`：I指示是否将Excel文档转换为HTML格式。默认为 `false` 。
        - `"delimiter"`：默认 `"\n!?。；！？"` 。
        - `"task_page_size"`：默认 `12` 。只对PDF生效。
        - `"raptor"`：使用召回增强RAPTOR策略。 默认: `{"use_raptor": false}` 。
    - 如果 `"chunk_method"` 是 `"qa"`，`"manuel"`，`"paper"`，`"book"`，`"laws"`，或者 `"presentation"`，`"parser_config"` 对象包含如下属性:
        - `"raptor"`：使用召回增强RAPTOR策略。 默认: `{"use_raptor": false}` 。
    - 如果 `"chunk_method"` 是 `"table"`，`"picture"`，`"one"`，或 `"email"`，`"parser_config"` 是一个空 JSON 对象。
    - 如果 `"chunk_method"` 是 `"knowledge_graph"`，`"parser_config"` 对象包含如下属性:
        - `"chunk_token_count"`：默认 `128` 。
        - `"delimiter"`：默认 `"\n!?。；！？"` 。
        - `"entity_types"`：默认 `["organization","person","location","event","time"]`

### Response

Success:

```
{
    "code": 0,
    "data": {
```

```
        "avatar": null,
        "chunk_count": 0,
        "chunk_method": "naive",
        "create_date": "Thu, 24 Oct 2024 09:14:07 GMT",
        "create_time": 1729761247434,
        "created_by": "69736c5e723611efb51b0242ac120007",
        "description": null,
        "document_count": 0,
        "embedding_model": "BAAI/bge-large-zh-v1.5",
        "id": "527fa74891e811ef9c650242ac120006",
        "language": "English",
        "name": "test_1",
        "parser_config": {
            "chunk_token_num": 128,
            "delimiter": "\\n!?;。；！？",
            "html4excel": false,
            "layout_recognize": true,
            "raptor": {
                "user_raptor": false
            }
        },
        "permission": "me",
        "similarity_threshold": 0.2,
        "status": "1",
        "tenant_id": "69736c5e723611efb51b0242ac120007",
        "token_num": 0,
        "update_date": "Thu, 24 Oct 2024 09:14:07 GMT",
        "update_time": 1729761247434,
        "vector_similarity_weight": 0.3
    }
}
```


Failure:

```
{
    "code": 102,
    "message": "Duplicated knowledgebase name in creating dataset."
}
```

## 删除知识库

**DELETE** `/api/v1/datasets`

用ID删除知识库。

### Request

- Method: DELETE
- URL: `/api/v1/datasets`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
  - Body:
    - `"ids"：list[string]`

#### Request example

```
curl --request DELETE \
     --url http://{address}/api/v1/datasets \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '{
```

```
     "ids": ["test_1", "test_2"]
     }'
```

#### Request parameters

- `"ids"`: (*Body parameter*), `list[string]`
  要删除的知识库的ID。如果未指定，则将删除所有知识库。

### Response

Success:

```
{
    "code": 0
}
```

Failure:

```
{
    "code": 102,
    "message": "You don't own the dataset."
}
```

## 更新知识库

**PUT** `/api/v1/datasets/{dataset_id}`

更新指定知识库的配置。

### Request

- Method: PUT
- URL: `/api/v1/datasets/{dataset_id}`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"name"`: `string`
  - `"embedding_model"`: `string`
  - `"chunk_method"`: `enum<string>`

#### Request example

```
curl --request PUT \
     --url http://{address}/api/v1/datasets/{dataset_id} \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '
     {
          "name": "updated_dataset"
     }'
```

#### Request parameters

- `dataset_id`: (*Path parameter*)
  要更新的知识库的ID。
- `"name"`: (*Body parameter*), `string`
  知识库的修订名称。

- `"embedding_model"`: (*Body parameter*), `string`
  更新的嵌入模型名称。
  - 确保 `"chunk_count"` is `0`， 在更新 `"embedding_model"` 之前。
- `"chunk_method"`: (*Body parameter*), `enum<string>`
  知识库的分块方法。可用选项:
  - `"naive"`: 通用（默认）
  - `"manual"`: 手册
  - `"qa"`: Q&A
  - `"table"`: 表格
  - `"paper"`: 论文
  - `"book"`: 书籍
  - `"laws"`: 法律
  - `"presentation"`: 演示文稿
  - `"picture"`: 图片
  - `"one"`: 不做切分的文档

### Response

Success:

```
{
    "code": 0
}
```

Failure:

```
{
    "code": 102,
```

```
    "message": "Can't change tenant_id."
}
```

## 列出知识库

**GET** `/api/v1/datasets?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&name={dataset_name}&id={dataset_id}`

列出知识库。

### Request

- Method: GET
- URL: `/api/v1/datasets?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&name={dataset_name}&id={dataset_id}`
- Headers:
  - `'Authorization: Bearer <YOUR_API_KEY>'`

#### Request example

```
curl --request GET \
     --url http://{address}/api/v1/datasets?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&name={dataset_name}&id={datase
     --header 'Authorization: Bearer <YOUR_API_KEY>'
```

#### Request parameters

- `page`: (*Filter parameter*)
  指定显示知识库的页面。默认 `1`。
- `page_size`: (*Filter parameter*)
  每页上的知识库数量。默认 `30`。

- `orderby` : (*Filter parameter*)
  知识库应排序的字段。可用选项:
  - `create_time` (default)
  - `update_time`
- `desc` : (*Filter parameter*)
  指示是否应按降序对检索到的知识库进行排序。 默认 `true` 。
- `name` : (*Filter parameter*)
  要检索的知识库的名称。
- `id` : (*Filter parameter*)
  要检索的知识库的ID。

### Response

Success:


```
{
    "code": 0,
    "data": [
        {
            "avatar": "",
            "chunk_count": 59,
            "create_date": "Sat, 14 Sep 2024 01:12:37 GMT",
            "create_time": 1726276357324,
            "created_by": "69736c5e723611efb51b0242ac120007",
            "description": null,
            "document_count": 1,
            "embedding_model": "BAAI/bge-large-zh-v1.5",
            "id": "6e211ee0723611efa10a0242ac120007",
            "language": "English",
            "name": "mysql",
            "chunk_method": "knowledge_graph",
            "parser_config": {
```

```
                "chunk_token_num": 8192,
                "delimiter": "\\n!?;。；！？",
                "entity_types": [
                    "organization",
                    "person",
                    "location",
                    "event",
                    "time"
                ]
            },
            "permission": "me",
            "similarity_threshold": 0.2,
            "status": "1",
            "tenant_id": "69736c5e723611efb51b0242ac120007",
            "token_num": 12744,
            "update_date": "Thu, 10 Oct 2024 04:07:23 GMT",
            "update_time": 1728533243536,
            "vector_similarity_weight": 0.3
        }
    ]
}
```


Failure:


```
{
    "code": 102,
    "message": "The dataset doesn't exist"
}
```


## 知识库文档管理

# 上传文档

**POST** `/api/v1/datasets/{dataset_id}/documents`

上传文档到具体知识库

### Request

- Method: POST
- URL: `/api/v1/datasets/{dataset_id}/documents`
- Headers:
  - `'Content-Type: multipart/form-data'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Form:
  - `'file=@{FILE_PATH}'`

#### Request example

```
curl --request POST \
     --url http://{address}/api/v1/datasets/{dataset_id}/documents \
     --header 'Content-Type: multipart/form-data' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --form 'file=@./test1.txt' \
     --form 'file=@./test2.pdf'
```

#### Request parameters

- `dataset_id` : (*Path parameter*)
  知识库ID
- `'file'` : (*Body parameter*)
  文档

### Response

Success:


```
{
    "code": 0,
    "data": [
        {
            "chunk_method": "naive",
            "created_by": "69736c5e723611efb51b0242ac120007",
            "dataset_id": "527fa74891e811ef9c650242ac120006",
            "id": "b330ec2e91ec11efbc510242ac120004",
            "location": "1.txt",
            "name": "1.txt",
            "parser_config": {
                "chunk_token_num": 128,
                "delimiter": "\\n!?;。；！？",
                "html4excel": false,
                "layout_recognize": true,
                "raptor": {
                    "user_raptor": false
                }
            },
            "run": "UNSTART",
            "size": 17966,
            "thumbnail": "",
            "type": "doc"
        }
    ]
}
```


Failure:

```
{
    "code": 101,
    "message": "No file part!"
}
```

## 更新文档

**PUT** `/api/v1/datasets/{dataset_id}/documents/{document_id}`

更新文档配置

### Request

- Method: PUT
- URL: `/api/v1/datasets/{dataset_id}/documents/{document_id}`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"name"`：`string`
  - `"chunk_method"`：`string`
  - `"parser_config"`：`object`

#### Request example

```
curl --request PUT \
     --url http://{address}/api/v1/datasets/{dataset_id}/info/{document_id} \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
```

```
--header 'Content-Type: application/json' \
--data '
{
     "name": "manual.txt",
     "chunk_method": "manual",
     "parser_config": {"chunk_token_count": 128}
}'
```

#### Request parameters

- `dataset_id`: (*Path parameter*)
  知识库ID
- `document_id`: (*Path parameter*)
  文档ID
- `"name"`: (*Body parameter*), `string`
- `"chunk_method"`: (*Body parameter*), `string`
  知识库的分块方法。可用选项:
  - `"naive"`: 通用（默认）
  - `"manual"`: 手册
  - `"qa"`: Q&A
  - `"table"`: 表格
  - `"paper"`: 论文
  - `"book"`: 书籍
  - `"laws"`: 法律
  - `"presentation"`: 演示文稿
  - `"picture"`: 图片
  - `"one"`: 不做切分的文档

- `"parser_config"`: (*Body parameter*), `object`
  知识库解析器的配置设置。此JSON对象中的属性随选择的 `"chunk_method"` 而定:
  - 如果 `"chunk_method"` 是 `"naive"`, `"parser_config"` 对象包含如下属性:
    - `"chunk_token_count"`: 默认 `128` 。
    - `"layout_recognize"`: 默认 `true` 。
    - `"html4excel"`: 指示是否将Excel文档转换为HTML格式。 默认 `false` 。
    - `"delimiter"`: 默认 `"\n!?。；！？"` 。
    - `"task_page_size"`: 默认 `12` . 尽对PDF生效。
    - `"raptor"`: 使用召回增强RAPTOR策略。 默认: `{"use_raptor": false}` 。
  - 如果 `"chunk_method"` 是 `"qa"`, `"manuel"`, `"paper"`, `"book"`, `"laws"`, 或者 `"presentation"`, `"parser_config"` 对象包含如下属性:
    - `"raptor"`: 使用召回增强RAPTOR策略。 默认: `{"use_raptor": false}` 。
  - 如果 `"chunk_method"` 是 `"table"`, `"picture"`, `"one"`, or `"email"`, `"parser_config"` 是一个空的JSON对象。
  - 如果 `"chunk_method"` 是 `"knowledge_graph"`, `"parser_config"` 对象包含如下属性:
    - `"chunk_token_count"`: 默认 `128` 。
    - `"delimiter"`: 默认 `"\n!?。；！？"` 。
    - `"entity_types"`: 默认 `["organization","person","location","event","time"]`

### Response

Success:

```
{
    "code": 0
}
```

Failure:

```
{
    "code": 102,
    "message": "The dataset does not have the document."
}
```

## 下载文档

**GET** `/api/v1/datasets/{dataset_id}/documents/{document_id}`

从知识库下载文档

### Request

- Method: GET
- URL: `/api/v1/datasets/{dataset_id}/documents/{document_id}`
- Headers:
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Output:
  - `'{PATH_TO_THE_FILE}'`

#### Request example

```
curl --request GET \
     --url http://{address}/api/v1/datasets/{dataset_id}/documents/{document_id} \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --output ./ZS AI Platform.txt
```

#### Request parameters

- `dataset_id`: (*Path parameter*)
  知识库ID。
- `documents_id`: (*Path parameter*)
  文档ID。

#### Response

Success:

```
This is a test to verify the file download feature.
```

Failure:

```
{
    "code": 102,
    "message": "You do not own the dataset 7898da028a0511efbf750242ac1220005."
}
```

## 列出文档

**GET** `/api/v1/datasets/{dataset_id}/documents?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&keywords={keywords}&id={document_id}&name={document_name}`

列出知识库文档。

#### Request

- Method: GET

- URL: `/api/v1/datasets/{dataset_id}/documents?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&keywords={keywords}&id={document_id}&name={document_name}`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`

#### Request example

```
curl --request GET \
     --url http://{address}/api/v1/datasets/{dataset_id}/documents?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&keyword
     --header 'Authorization: Bearer <YOUR_API_KEY>'
```

#### Request parameters

- `dataset_id`: (*Path parameter*)
  知识库ID。
- `keywords`: (*Filter parameter*), `string`
  文档名关键词。
- `page`: (*Filter parameter*), `integer` 指定显示文档的页面。 默认 `1`.
- `page_size`: (*Filter parameter*), `integer`
  每页上的最大文档数。 默认 `30`.
- `orderby`: (*Filter parameter*), `string`
  文档排序的字段。可用选项:
  - `create_time` (default)
  - `update_time`
- `desc`: (*Filter parameter*), `boolean`
  指示检索到的文档是否应按降序排序。 默认 `true`.

- `id` : (*Filter parameter*), `string`
  要检索的文档的ID。

### Response

Success:


```
{
    "code": 0,
    "data": {
        "docs": [
            {
                "chunk_count": 0,
                "create_date": "Mon, 14 Oct 2024 09:11:01 GMT",
                "create_time": 1728897061948,
                "created_by": "69736c5e723611efb51b0242ac120007",
                "id": "3bcfbf8a8a0c11ef8aba0242ac120006",
                "knowledgebase_id": "7898da028a0511efbf750242ac120005",
                "location": "Test_2.txt",
                "name": "Test_2.txt",
                "parser_config": {
                    "chunk_token_count": 128,
                    "delimiter": "\n!?。；！？",
                    "layout_recognize": true,
                    "task_page_size": 12
                },
                "chunk_method": "naive",
                "process_begin_at": null,
                "process_duation": 0.0,
                "progress": 0.0,
                "progress_msg": "",
                "run": "0",
                "size": 7,
                "source_type": "local",
                "status": "1",
```

```
                "thumbnail": null,
                "token_count": 0,
                "type": "doc",
                "update_date": "Mon, 14 Oct 2024 09:11:01 GMT",
                "update_time": 1728897061948
            }
        ],
        "total": 1
    }
}
```


Failure:


```
{
    "code": 102,
    "message": "You don't own the dataset 7898da028a0511efbf750242ac1220005. "
}
```


## 删除文档

**DELETE** `/api/v1/datasets/{dataset_id}/documents`

按ID删除文档。

### Request

- Method: DELETE
- URL: `/api/v1/datasets/{dataset_id}/documents`
- Headers:
  - `'Content-Type: application/json'`

    - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
    - `"ids"：list[string]`

#### Request example

```
curl --request DELETE \
     --url http://{address}/api/v1/datasets/{dataset_id}/documents \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '
     {
          "ids": ["id_1","id_2"]
     }'
```

#### Request parameters

- `dataset_id`：(*Path parameter*)
  知识库ID。
- `"ids"`：(*Body parameter*)，`list[string]`
  要删除的文档的ID。如果未指定，则将删除指定知识库中的所有文档。

### Response

Success:

```
{
    "code": 0
}.
```

Failure:

```
{
    "code": 102,
    "message": "You do not own the dataset 7898da028a0511efbf750242ac1220005."
}
```

## 解析文档

**POST** `/api/v1/datasets/{dataset_id}/chunks`

解析指定知识库中的文档。

### Request

- Method: POST
- URL: `/api/v1/datasets/{dataset_id}/chunks`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"document_ids"： list[string]`

#### Request example

```
curl --request POST \
     --url http://{address}/api/v1/datasets/{dataset_id}/chunks \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '
     {
```

```
     "document_ids": ["97a5f1c2759811efaa500242ac120004","97ad64b6759811ef9fc30242ac120004"]
}'
```

#### Request parameters

- `dataset_id`：(*Path parameter*)
  知识库ID。
- `"document_ids"`：(*Body parameter*)，`list[string]`，*Required*
  要解析的文档的ID。

### Response

Success:

```
{
    "code": 0
}
```

Failure:

```
{
    "code": 102,
    "message": "`document_ids` is required"
}
```

## 停止解析文档

**DELETE** `/api/v1/datasets/{dataset_id}/chunks`

停止解析指定的文档。

## Request

- Method: DELETE
- URL: `/api/v1/datasets/{dataset_id}/chunks`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"document_ids"`: `list[string]`

### Request example

```
curl --request DELETE \
     --url http://{address}/api/v1/datasets/{dataset_id}/chunks \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '
     {
          "document_ids": ["97a5f1c2759811efaa500242ac120004","97ad64b6759811ef9fc30242ac120004"]
     }'
```

### Request parameters

- `dataset_id`: (*Path parameter*)
  知识库ID。
- `"document_ids"`: (*Body parameter*), `list[string]`, *Required*
  应停止解析的文档的ID。

## Response

Success:

```
{
    "code": 0
}
```

Failure:

```
{
    "code": 102,
    "message": "`document_ids` is required"
}
```

# 知识库中的块管理

## Add chunk

**POST** `/api/v1/datasets/{dataset_id}/documents/{document_id}/chunks`

将块添加到指定知识库中的指定文档中。

### Request

- Method: POST
- URL: `/api/v1/datasets/{dataset_id}/documents/{document_id}/chunks`
- Headers:
  - `'content-Type: application/json'`

    - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
    - `"content"：string`
    - `"important_keywords"：list[string]`

#### Request example

```
curl --request POST \
     --url http://{address}/api/v1/datasets/{dataset_id}/documents/{document_id}/chunks \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '
     {
          "content": "<CHUNK_CONTENT_HERE>"
     }'
```

#### Request parameters

- `dataset_id`：(*Path parameter*)
  关联的知识库ID。
- `document_ids`：(*Path parameter*)
  关联的文档ID。
- `"content"`：(*Body parameter*)，`string`，*Required*
  块的文本内容。
- `"important_keywords` (*Body parameter*)，`list[string]`
  用块标记的关键术语或短语。
- `"questions"` (*Body parameter*)，`list[string]` 如果有一个给定的问题，嵌入的块将基于它们。

### Response

Success:


```
{
    "code": 0,
    "data": {
        "chunk": {
            "content": "who are you",
            "create_time": "2024-12-30 16:59:55",
            "create_timestamp": 1735549195.969164,
            "dataset_id": "72f36e1ebdf411efb7250242ac120006",
            "document_id": "61d68474be0111ef98dd0242ac120006",
            "id": "12ccdc56e59837e5",
            "important_keywords": [],
            "questions": []
        }
    }
}
```


Failure:


```
{
    "code": 102,
    "message": "`content` is required"
}
```


## 列出块

**GET** `/api/v1/datasets/{dataset_id}/documents/{document_id}/chunks?keywords={keywords}&page={page}&page_size={page_size}&id={id}`

列出指定文档中的块。

## Request

- Method: GET
- URL: `/api/v1/datasets/{dataset_id}/documents/{document_id}/chunks?keywords={keywords}&page={page}&page_size={page_size}&id={chunk_id}`
- Headers:
  - `'Authorization: Bearer <YOUR_API_KEY>'`

### Request example

```
curl --request GET \
     --url http://{address}/api/v1/datasets/{dataset_id}/documents/{document_id}/chunks?keywords={keywords}&page={page}&page_size={page_si
     --header 'Authorization: Bearer <YOUR_API_KEY>'
```

### Request parameters

- `dataset_id`: (*Path parameter*)
  关联的知识库ID。
- `document_ids`: (*Path parameter*)
  关联的文档ID。
- `keywords` (*Filter parameter*), `string`
  用于匹配块内容的关键字。
- `page` (*Filter parameter*), `integer`
  指定显示块的页面。 默认 `1`.
- `page_size` (*Filter parameter*), `integer`
  每页上的最大块数。 默认 `1024`.
- `id` (*Filter parameter*), `string`
  要检索的块的ID。

## Response

Success:


```
{
    "code": 0,
    "data": {
        "chunks": [
            {
                "available_int": 1,
                "content": "This is a test content.",
                "docnm_kwd": "1.txt",
                "document_id": "b330ec2e91ec11efbc510242ac120004",
                "id": "b48c170e90f70af998485c1065490726",
                "image_id": "",
                "important_keywords": "",
                "positions": [
                    ""
                ]
            }
        ],
        "doc": {
            "chunk_count": 1,
            "chunk_method": "naive",
            "create_date": "Thu, 24 Oct 2024 09:45:27 GMT",
            "create_time": 1729763127646,
            "created_by": "69736c5e723611efb51b0242ac120007",
            "dataset_id": "527fa74891e811ef9c650242ac120006",
            "id": "b330ec2e91ec11efbc510242ac120004",
            "location": "1.txt",
            "name": "1.txt",
            "parser_config": {
                "chunk_token_num": 128,
                "delimiter": "\\n!?;。；！？",
                "html4excel": false,
                "layout_recognize": true,
                "raptor": {
```

```
                    "user_raptor": false
                }
            },
            "process_begin_at": "Thu, 24 Oct 2024 09:56:44 GMT",
            "process_duation": 0.54213,
            "progress": 0.0,
            "progress_msg": "Task dispatched...",
            "run": "2",
            "size": 17966,
            "source_type": "local",
            "status": "1",
            "thumbnail": "",
            "token_count": 8,
            "type": "doc",
            "update_date": "Thu, 24 Oct 2024 11:03:15 GMT",
            "update_time": 1729767795721
        },
        "total": 1
    }
}
```


Failure:


```
{
    "code": 102,
    "message": "You don't own the document 5c5999ec7be811ef9cab0242ac12000e5."
}
```


## 删除块

**DELETE** `/api/v1/datasets/{dataset_id}/documents/{document_id}/chunks`

Deletes chunks by ID.

### Request

- Method: DELETE
- URL: `/api/v1/datasets/{dataset_id}/documents/{document_id}/chunks`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"chunk_ids"：list[string]`

#### Request example

```
curl --request DELETE \
     --url http://{address}/api/v1/datasets/{dataset_id}/documents/{document_id}/chunks \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '
     {
          "chunk_ids": ["test_1", "test_2"]
     }'
```

#### Request parameters

- `dataset_id`：(*Path parameter*)
  关联的知识库ID。
- `document_ids`：(*Path parameter*)
  关联的文档ID。

- `"chunk_ids"`：(*Body parameter*)，`list[string]`
  要删除的块的ID。如果未指定，则将删除指定文档的所有块。

### Response

Success:

```
{
    "code": 0
}
```

Failure:

```
{
    "code": 102,
    "message": "`chunk_ids` is required"
}
```

## 更新块

**PUT** `/api/v1/datasets/{dataset_id}/documents/{document_id}/chunks/{chunk_id}`

更新指定块的内容或配置。

### Request

- Method: PUT
- URL: `/api/v1/datasets/{dataset_id}/documents/{document_id}/chunks/{chunk_id}`
- Headers:

    - 'content-Type: application/json'
    - 'Authorization: Bearer <YOUR_API_KEY>'
- Body:
    - "content"：string
    - "important_keywords"：list[string]
    - "available"：boolean

#### Request example

```
curl --request PUT \
     --url http://{address}/api/v1/datasets/{dataset_id}/documents/{document_id}/chunks/{chunk_id} \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '
     {
          "content": "ZS AI Platform 123",
          "important_keywords": []
     }'
```

#### Request parameters

- dataset_id：(*Path parameter*)
  关联的知识库ID。
- document_ids：(*Path parameter*)
  关联的文档ID。
- chunk_id：(*Path parameter*)
  要更新的块的ID。
- "content"：(*Body parameter*), string
  块的文本内容。

- `"important_keywords"`：(*Body parameter*), `list[string]`
  要用块标记的关键术语或短语列表。
- `"available"`：(*Body parameter*) `boolean`
  知识库中块的可用性状态。值选项：
  - `true`：可用 (default)
  - `false`：不可用

#### Response

Success:

```
{
    "code": 0
}
```

Failure:

```
{
    "code": 102,
    "message": "Can't find this chunk 29a2d9987e16ba331fb4d7d30d99b71d2"
}
```

## 检索块

**POST** `/api/v1/retrieval`

从指定的知识库中检索块。

#### Request

- Method: POST
- URL: `/api/v1/retrieval`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"question"：string`
  - `"dataset_ids"：list[string]`
  - `"document_ids"：list[string]`
  - `"page"：integer`
  - `"page_size"：integer`
  - `"similarity_threshold"：float`
  - `"vector_similarity_weight"：float`
  - `"top_k"：integer`
  - `"rerank_id"：string`
  - `"keyword"：boolean`
  - `"highlight"：boolean`

**Request example**

```
curl --request POST \
     --url http://{address}/api/v1/retrieval \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '
     {
          "question": "What is advantage of ZS AI Platform?",
          "dataset_ids": ["b2a62730759d11ef987d0242ac120004"],
```

```
        "document_ids": ["77df9ef4759a11ef8bdd0242ac120004"]
    }'
```

**Request parameter**

- `"question"`: (*Body parameter*), `string`, *Required*
  用户查询或查询关键字。
- `"dataset_ids"`: (*Body parameter*) `list[string]`
  要搜索的知识库的ID。如果不设置此参数，请确保已设置 `"document_ids"`.
- `"document_ids"`: (*Body parameter*), `list[string]`
  要搜索的文档的ID。确保所有选定的文档使用相同的嵌入模型。否则，将出现错误。如果不设置此参数，请确保已设置 `"dataset_ids"`.
- `"page"`: (*Body parameter*), `integer`
  指定显示块的页面。 默认 `1`.
- `"page_size"`: (*Body parameter*)
  每页上的最大块数。 默认 `30`.
- `"similarity_threshold"`: (*Body parameter*)
  最小相似性得分。默认 `0.2`.
- `"vector_similarity_weight"`: (*Body parameter*), `float`
  向量余弦相似度的权重。 默认 `0.3` .如果x表示向量余弦相似度的权重，则（1-x）是术语相似度权重。
- `"top_k"`: (*Body parameter*), `integer`
  进行向量余弦计算的块数。 默认 `1024`.
- `"rerank_id"`: (*Body parameter*), `integer`
  重新rerank模型的ID。
- `"keyword"`: (*Body parameter*), `boolean`
  指示是否启用基于关键字的匹配：
  - `true`：启用基于关键字的匹配。
  - `false`：禁用基于关键字的匹配（默认）。

- `"highlight"`: (*Body parameter*), `boolean`
  指定是否在结果中突出显示匹配的术语：
  - `true`：启用匹配术语的突出显示。
  - `false`：禁用突出显示匹配的术语（默认）。

### Response

Success:


```
{
    "code": 0,
    "data": {
        "chunks": [
            {
                "content": "ZS AI Platform content",
                "content_ltks": "ZS AI Platform content",
                "document_id": "5c5999ec7be811ef9cab0242ac120005",
                "document_keyword": "1.txt",
                "highlight": "<em>ZS AI Platform</em> content",
                "id": "d78435d142bd5cf6704da62c778795c5",
                "image_id": "",
                "important_keywords": [
                    ""
                ],
                "kb_id": "c7ee74067a2c11efb21c0242ac120006",
                "positions": [
                    ""
                ],
                "similarity": 0.9669436601210759,
                "term_similarity": 1.0,
                "vector_similarity": 0.8898122004035864
            }
        ],
        "doc_aggs": [
```

```
            {
                "count": 1,
                "doc_id": "5c5999ec7be811ef9cab0242ac120005",
                "doc_name": "1.txt"
            }
        ],
        "total": 1
    }
}
```


Failure:


```
{
    "code": 102,
    "message": "`datasets` is required."
}
```


# 聊天助手管理

## 创建聊天助手

**POST** `/api/v1/chats`

创建聊天助手。

### Request

- Method: POST

- URL: `/api/v1/chats`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"name"： string`
  - `"avatar"： string`
  - `"dataset_ids"： list[string]`
  - `"llm"： object`
  - `"prompt"： object`

#### Request example

```
curl --request POST \
     --url http://{address}/api/v1/chats \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '{
    "dataset_ids": ["0b2cbc8c877f11ef89070242ac120005"],
    "name":"new_chat_1"
}'
```

#### Request parameters

- `"name"`：(*Body parameter*)，`string`，*Required*
  聊天助理的名称。
- `"avatar"`：(*Body parameter*)，`string`
  Base64编码。

- `"dataset_ids"`：(*Body parameter*)，`list[string]`
  关联知识库的ID。
- `"llm"`：(*Body parameter*)，`object`
  聊天助手要创建的LLM设置。如果没有明确设置，将生成具有以下值的JSON对象作为默认值。一个 `llm` JSON对象包含以下属性：
  - `"model_name"`，`string`
    聊天模型名称。如果未设置，将使用用户的默认聊天模式。
  - `"temperature"`：`float`
    控制模型预测的随机性。较低的温度会导致更保守的反应，而较高的温度会产生更具创造性和多样性的反应。 默认 `0.1`。
  - `"top_p"`：`float`
    也称为"核采样"，此参数设置一个阈值，以选择一组较小的单词进行采样。它关注最可能的单词，剪掉不太可能的单词。 默认 `0.3`
  - `"presence_penalty"`：`float`
    这会通过惩罚对话中已经出现的单词来阻止模型重复相同的信息。 默认 `0.2`。
  - `"frequency penalty"`：`float`
    与 `presence penalty` 类似，这降低了模型频繁重复相同单词的倾向。 默认 `0.7`。
  - `"max_token"`：`integer`
    模型输出的最大长度，以标记（单词或单词片段）的数量来衡量。默认 `512`。如果禁用，则提高最大token限制，允许模型确定其响应中的token数量。
- `"prompt"`：(*Body parameter*)，`object`
  LLM应遵循的说明。如果没有明确设置，将生成具有以下值的JSON对象作为默认值。 `prompt` JSON对象包含以下属性：
  - `"similarity_threshold"`：`float` ZS AI Platform 在检索过程中采用加权关键字相似度和加权向量余弦相似度的组合或加权关键字相似度与加权重新排序分数的组合。此参数设置了用户查询和块之间相似性的阈值。如果相似性得分低于此阈值，则相应的块将从结果中排除。默认值为 `0.2`。
  - `"keywords_similarity_weight"`：`float` 该参数通过向量余弦相似性或重新排序模型相似性来设置混合相似性得分中关键字相似性的权重。通过调整此权重，您可以控制关键字相似性对其他相似性度量的影响。默认值为 `0.7`。
  - `"top_n"`：`int` 此参数指定了提供给LLM的相似度得分高于 `similarity_threshold` 的头部块的数量。LLM将*仅*访问这些 `top N` 块。默认值为 `8`。
  - `"variables"`：`object[]` 此参数列出了在**聊天配置**的"系统"字段中使用的变量。请注意：
    - `"knowledge"` 是一个保留变量，表示检索到的块。

        - 'System'中的所有变量都应该用花括号括起来。
        - 默认值为 `[{"key": "knowledge", "optional": true}]` 。
    - `"rerank_model"`： `string` 如果没有指定，将使用向量余弦相似度；否则，将使用rerank score。
    - `top_k`： `int` 是指根据特定的排名标准从列表或集合中重新排序或选择前k个项目的过程。默认为 1024.
    - `"empty_response"`： `string` 如果在知识库中没有检索到用户问题的任何内容，则将其用作响应。为了让LLM在一无所获时即兴发挥，请留空。
    - `"opener"`： `string` 对用户的开场问候. 默认 `"Hi! I am your assistant, can I help you?"` .
    - `"show_quote`： `boolean` 指示是否应显示文本源. 默认 `true` .
    - `"prompt"`： `string` 提示内容。

### Response

Success:


```
{
    "code": 0,
    "data": {
        "avatar": "",
        "create_date": "Thu, 24 Oct 2024 11:18:29 GMT",
        "create_time": 1729768709023,
        "dataset_ids": [
            "527fa74891e811ef9c650242ac120006"
        ],
        "description": "A helpful Assistant",
        "do_refer": "1",
        "id": "b1f2f15691f911ef81180242ac120003",
        "language": "English",
        "llm": {
            "frequency_penalty": 0.7,
            "max_tokens": 512,
            "model_name": "qwen-plus@Tongyi-Qianwen",
            "presence_penalty": 0.4,
```

```
            "temperature": 0.1,
            "top_p": 0.3
        },
        "name": "12234",
        "prompt": {
            "empty_response": "Sorry! No relevant content was found in the knowledge base!",
            "keywords_similarity_weight": 0.3,
            "opener": "Hi! I'm your assistant, what can I do for you?",
            "prompt": "You are an intelligent assistant. Please summarize the content of the knowledge base to answer the question. Please
            "rerank_model": "",
            "similarity_threshold": 0.2,
            "top_n": 6,
            "variables": [
                {
                    "key": "knowledge",
                    "optional": false
                }
            ]
        },
        "prompt_type": "simple",
        "status": "1",
        "tenant_id": "69736c5e723611efb51b0242ac120007",
        "top_k": 1024,
        "update_date": "Thu, 24 Oct 2024 11:18:29 GMT",
        "update_time": 1729768709023
    }
}
```


Failure:


```
{
    "code": 102,
    "message": "Duplicated chat name in creating dataset."
}
```

## 更新聊天助手

**PUT** `/api/v1/chats/{chat_id}`

更新指定聊天助手的配置。

### Request

- Method: PUT
- URL: `/api/v1/chats/{chat_id}`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"name"： string`
  - `"avatar"： string`
  - `"dataset_ids"： list[string]`
  - `"llm"： object`
  - `"prompt"： object`

#### Request example

```
curl --request PUT \
     --url http://{address}/api/v1/chats/{chat_id} \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '
     {
```

```
        "name":"Test"
    }'
```

### Parameters

- `chat_id`：(*Path parameter*)
  要更新的聊天助手的ID。
- `"name"`：(*Body parameter*)，`string`，*Required*
  聊天助理的名称。
- `"avatar"`：(*Body parameter*)，`string`
  Base64编码。
- `"dataset_ids"`：(*Body parameter*)，`list[string]`
  关联知识库的ID。
- `"llm"`：(*Body parameter*)，`object`
  聊天助手要创建的LLM设置。如果没有明确设置，将生成具有以下值的JSON对象作为默认值。一个 `llm` JSON对象包含以下属性：
  - `"model_name"`，`string`
    聊天模型名称。如果未设置，将使用用户的默认聊天模式。
  - `"temperature"`：`float`
    控制模型预测的随机性。较低的温度会导致更保守的反应，而较高的温度会产生更具创造性和多样性的反应。 默认 `0.1`。
  - `"top_p"`：`float`
    也称为"核采样"，此参数设置一个阈值，以选择一组较小的单词进行采样。它关注最可能的单词，剪掉不太可能的单词。 默认 `0.3`
  - `"presence_penalty"`：`float`
    这会通过惩罚对话中已经出现的单词来阻止模型重复相同的信息。 默认 `0.2`。
  - `"frequency penalty"`：`float`
    与 `presence penalty` 类似，这降低了模型频繁重复相同单词的倾向。 默认 `0.7`。
  - `"max_token"`：`integer`
    模型输出的最大长度，以标记（单词或单词片段）的数量来衡量。默认 `512`。如果禁用，则提高最大token限制，允许模型确定其响应中的token数量。

- `"prompt"`: (*Body parameter*), `object`
  LLM应遵循的说明。如果没有明确设置，将生成具有以下值的JSON对象作为默认值。 `prompt` JSON对象包含以下属性：
  - `"similarity_threshold"`： `float` ZS AI Platform 在检索过程中采用加权关键字相似度和加权向量余弦相似度的组合或加权关键字相似度与加权重新排序分数的组合。此参数设置了用户查询和块之间相似性的阈值。如果相似性得分低于此阈值，则相应的块将从结果中排除。默认值为 `0.2`.
  - `"keywords_similarity_weight"`： `float` 该参数通过向量余弦相似性或重新排序模型相似性来设置混合相似性得分中关键字相似性的权重。通过调整此权重，您可以控制关键字相似性对其他相似性度量的影响。默认值为 `0.7`.
  - `"top_n"`： `int` 此参数指定了提供给LLM的相似度得分高于 `similarity_threshold` 的头部块的数量。LLM将仅访问这些 `top N` 块。默认值为 `8`.
  - `"variables"`： `object[]` 此参数列出了在**聊天配置**的"系统"字段中使用的变量。请注意：
    - `"knowledge"` 是一个保留变量，表示检索到的块。
    - 'System'中的所有变量都应该用花括号括起来。
    - 默认值为 `[{"key": "knowledge", "optional": true}]`。
  - `"rerank_model"`： `string` 如果没有指定，将使用向量余弦相似度；否则，将使用rerank score。
  - `"empty_response"`： `string` 如果在知识库中没有检索到用户问题的任何内容，则将其用作响应。为了让LLM在一无所获时即兴发挥，请留空。
  - `"opener"`： `string` 对用户的开场问候. 默认 `"Hi! I am your assistant, can I help you?"`.
  - `"show_quote`： `boolean` 指示是否应显示文本源. 默认 `true`.
  - `"prompt"`： `string` 提示内容。

### Response

Success:

```
{
    "code": 0
}
```

Failure:

```
{
    "code": 102,
    "message": "Duplicated chat name in updating dataset."
}
```

## 删除聊天助手

**DELETE** `/api/v1/chats`

按ID删除聊天助理。

### Request

- Method: DELETE
- URL: `/api/v1/chats`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"ids"：list[string]`

#### Request example

```
curl --request DELETE \
     --url http://{address}/api/v1/chats \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
```

```
--data '
{
     "ids": ["test_1", "test_2"]
}'
```

#### Request parameters

- `"ids"`: (*Body parameter*), `list[string]`
  要删除的聊天助理的ID。如果未指定，系统中的所有聊天助手都将被删除。

### Response

Success:

```
{
    "code": 0
}
```

Failure:

```
{
    "code": 102,
    "message": "ids are required"
}
```

## 列出聊天助手

**GET** `/api/v1/chats?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&name={chat_name}&id={chat_id}`

列出聊天助手。

### Request

- Method: GET
- URL: `/api/v1/chats?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&name={dataset_name}&id={dataset_id}`
- Headers:
  - `'Authorization: Bearer <YOUR_API_KEY>'`

#### Request example

```
curl --request GET \
     --url http://{address}/api/v1/chats?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&name={dataset_name}&id={dataset_i
     --header 'Authorization: Bearer <YOUR_API_KEY>'
```

#### Request parameters

- `page`: (*Filter parameter*), `integer`
  指定显示聊天助手的页面。默认 `1`.
- `page_size`: (*Filter parameter*), `integer`
  每个页面上的聊天助理数量。 默认 `30`.
- `orderby`: (*Filter parameter*), `string`
  对结果进行排序的属性。可用选项:
  - `create_time` (default)
  - `update_time`
- `desc`: (*Filter parameter*), `boolean`
  指示是否应按降序对检索到的聊天助手进行排序。 默认 `true`.
- `id`: (*Filter parameter*), `string`
  要检索的聊天助手的ID。

- `name`: (*Filter parameter*), `string`
  要检索的聊天助手的名称。

### Response

Success:


```
{
    "code": 0,
    "data": [
        {
            "avatar": "",
            "create_date": "Fri, 18 Oct 2024 06:20:06 GMT",
            "create_time": 1729232406637,
            "description": "A helpful Assistant",
            "do_refer": "1",
            "id": "04d0d8e28d1911efa3630242ac120006",
            "dataset_ids": ["527fa74891e811ef9c650242ac120006"],
            "language": "English",
            "llm": {
                "frequency_penalty": 0.7,
                "max_tokens": 512,
                "model_name": "qwen-plus@Tongyi-Qianwen",
                "presence_penalty": 0.4,
                "temperature": 0.1,
                "top_p": 0.3
            },
            "name": "13243",
            "prompt": {
                "empty_response": "Sorry! No relevant content was found in the knowledge base!",
                "keywords_similarity_weight": 0.3,
                "opener": "Hi! I'm your assistant, what can I do for you?",
                "prompt": "You are an intelligent assistant. Please summarize the content of the knowledge base to answer the question. Pl
                "rerank_model": "",
                "similarity_threshold": 0.2,
```

```
                "top_n": 6,
                "variables": [
                    {
                        "key": "knowledge",
                        "optional": false
                    }
                ]
            },
            "prompt_type": "simple",
            "status": "1",
            "tenant_id": "69736c5e723611efb51b0242ac120007",
            "top_k": 1024,
            "update_date": "Fri, 18 Oct 2024 06:20:06 GMT",
            "update_time": 1729232406638
        }
    ]
}
```


Failure:


```
{
    "code": 102,
    "message": "The chat doesn't exist"
}
```


# 会话管理

## 使用聊天助手创建会话

**POST** `/api/v1/chats/{chat_id}/sessions`

使用聊天助手创建会话.

### Request

- Method: POST
- URL: `/api/v1/chats/{chat_id}/sessions`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"name"`: `string`
  - `"user_id"`: `string` (optional)

#### Request example

```
curl --request POST \
     --url http://{address}/api/v1/chats/{chat_id}/sessions \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '
     {
          "name": "new session"
     }'
```

#### Request parameters

- `chat_id`: (*Path parameter*)
  关联聊天助手的ID。

- `"name"` : (*Body parameter*), `string`
  要创建的聊天会话的名称。
- `"user_id"` : (*Body parameter*), `string`
  可选的用户定义ID。

### Response

Success:

```
{
    "code": 0,
    "data": {
        "chat_id": "2ca4b22e878011ef88fe0242ac120005",
        "create_date": "Fri, 11 Oct 2024 08:46:14 GMT",
        "create_time": 1728636374571,
        "id": "4606b4ec87ad11efbc4f0242ac120006",
        "messages": [
            {
                "content": "Hi! I am your assistant，can I help you?",
                "role": "assistant"
            }
        ],
        "name": "new session",
        "update_date": "Fri, 11 Oct 2024 08:46:14 GMT",
        "update_time": 1728636374571
    }
}
```

Failure:

```
{
    "code": 102,
```

```
    "message": "Name cannot be empty."
}
```

## 更新聊天助理的会话

**PUT** `/api/v1/chats/{chat_id}/sessions/{session_id}`

更新指定聊天助手的会话。

### Request

- Method: PUT
- URL: `/api/v1/chats/{chat_id}/sessions/{session_id}`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"name： string`
  - `"user_id： string` (optional)

#### Request example

```
curl --request PUT \
     --url http://{address}/api/v1/chats/{chat_id}/sessions/{session_id} \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '
     {
```

```
        "name": "<REVISED_SESSION_NAME_HERE>"
    }'
```

#### Request Parameter

- `chat_id`：(*Path parameter*)
  关联聊天助手的ID。
- `session_id`：(*Path parameter*)
  要更新的会话的ID。
- `"name"`：(*Body Parameter*)，`string`
  修订后的会议名称。
- `"user_id"`：(*Body parameter*)，`string`
  可选的用户定义ID。

### Response

Success:

```
{
    "code": 0
}
```

Failure:

```
{
    "code": 102,
    "message": "Name cannot be empty."
}
```

## 列出聊天助理的会话

GET `/api/v1/chats/{chat_id}/sessions?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&name={session_name}&id={session_id}`

列出与指定聊天助手关联的会话。

### Request

- Method: GET
- URL: `/api/v1/chats/{chat_id}/sessions?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&name={session_name}&id={session_id}&user_id={user_id}`
- Headers:
  - `'Authorization: Bearer <YOUR_API_KEY>'`

#### Request example

```
curl --request GET \
     --url http://{address}/api/v1/chats/{chat_id}/sessions?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&name={session_
     --header 'Authorization: Bearer <YOUR_API_KEY>'
```

#### Request Parameters

- `chat_id` : (*Path parameter*)
  关联聊天助手的ID。
- `page` : (*Filter parameter*), `integer`
  指定显示会话的页面。 默认 `1` .
- `page_size` : (*Filter parameter*), `integer`
  每页上的会话数。默认 `30` .
- `orderby` : (*Filter parameter*), `string`
  会话排序的字段。可用选项:

    - `create_time` (default)
    - `update_time`
- `desc` : (*Filter parameter*), `boolean`
  指示是否应按降序对检索到的会话进行排序。 默认 `true` .
- `name` : (*Filter parameter*) `string`
  要检索的聊天会话的名称。
- `id` : (*Filter parameter*), `string`
  要检索的聊天会话的ID。
- `user_id` : (*Filter parameter*), `string`
  创建会话时传入的可选用户定义ID。

### Response

Success:


```
{
    "code": 0,
    "data": [
        {
            "chat": "2ca4b22e878011ef88fe0242ac120005",
            "create_date": "Fri, 11 Oct 2024 08:46:43 GMT",
            "create_time": 1728636403974,
            "id": "578d541e87ad11ef96b90242ac120006",
            "messages": [
                {
                    "content": "Hi! I am your assistant, can I help you?",
                    "role": "assistant"
                }
            ],
            "name": "new session",
            "update_date": "Fri, 11 Oct 2024 08:46:43 GMT",
            "update_time": 1728636403974
```

```
        }
    ]
}
```

Failure:

```
{
    "code": 102,
    "message": "The session doesn't exist"
}
```

## 删除聊天助理的会话

**DELETE** `/api/v1/chats/{chat_id}/sessions`

按ID删除聊天助手的会话。

### Request

- Method: DELETE
- URL: `/api/v1/chats/{chat_id}/sessions`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"ids"：list[string]`

#### Request example

```
curl --request DELETE \
     --url http://{address}/api/v1/chats/{chat_id}/sessions \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '
     {
          "ids": ["test_1", "test_2"]
     }'
```

#### Request Parameters

- `chat_id`: (*Path parameter*)
  关联聊天助手的ID。
- `"ids"`: (*Body Parameter*), `list[string]`
  要删除的会话的ID。如果未指定，则将删除与指定聊天助手关联的所有会话。

### Response

Success:

```
{
    "code": 0
}
```

Failure:

```
{
    "code": 102,
    "message": "The chat doesn't own the session"
}
```

## 与聊天助手交谈

**POST** `/api/v1/chats/{chat_id}/completions`

向指定的聊天助手提出一个问题，以开始人工智能驱动的对话。

:::tip NOTE

- 在流模式下，并非所有响应都包含引用，因为这取决于系统的判断。
- 在流模式下，最后一条消息是空消息：

  ```
  data:
  {
    "code": 0,
    "data": true
  }
  ```

:::

### Request

- Method: POST
- URL: `/api/v1/chats/{chat_id}/completions`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"question"`：`string`
  - `"stream"`：`boolean`
  - `"session_id"`：`string` (optional)

- "user_id：string（optional）

**Request example**

```
curl --request POST \
     --url http://{address}/api/v1/chats/{chat_id}/completions \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data-binary '
     {
     }'
```

```
curl --request POST \
     --url http://{address}/api/v1/chats/{chat_id}/completions \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data-binary '
     {
          "question": "Who are you",
          "stream": true,
          "session_id":"9fa7691cb85c11ef9c5f0242ac120005"
     }'
```

**Request Parameters**

- chat_id：(*Path parameter*)
  关联聊天助手的ID。
- "question"：(*Body Parameter*)，string，*Required*
  开始人工智能对话的问题。
- "stream"：(*Body Parameter*)，boolean
  指示是否以流式方式输出响应：

    - `true`：启用流（默认）。
    - `false`：不启用流
- `"session_id"`：(*Body Parameter*)
  会话的ID。如果没有提供，将生成一个新的会话。
- `"user_id"`：(*Body parameter*), `string`
  可选的用户定义ID。仅在未提供 `session_ID` 时有效。

### Response

Success without `session_id`：

```
data:{
    "code": 0,
    "message": "",
    "data": {
        "answer": "Hi! I'm your assistant, what can I do for you?",
        "reference": {},
        "audio_binary": null,
        "id": null,
        "session_id": "b01eed84b85611efa0e90242ac120005"
    }
}
data:{
    "code": 0,
    "message": "",
    "data": true
}
```

Success with `session_id`：

```
data:{
    "code": 0,
```

```
    "data": {
        "answer": "I am an intelligent assistant designed to help answer questions by summarizing content from a",
        "reference": {},
        "audio_binary": null,
        "id": "a84c5dd4-97b4-4624-8c3b-974012c8000d",
        "session_id": "82b0ab2a9c1911ef9d870242ac120006"
    }
}
data:{
    "code": 0,
    "data": {
        "answer": "I am an intelligent assistant designed to help answer questions by summarizing content from a knowledge base. My respon
        "reference": {},
        "audio_binary": null,
        "id": "a84c5dd4-97b4-4624-8c3b-974012c8000d",
        "session_id": "82b0ab2a9c1911ef9d870242ac120006"
    }
}
data:{
    "code": 0,
    "data": {
        "answer": "I am an intelligent assistant designed to help answer questions by summarizing content from a knowledge base. My respon
        "reference": {},
        "audio_binary": null,
        "id": "a84c5dd4-97b4-4624-8c3b-974012c8000d",
        "session_id": "82b0ab2a9c1911ef9d870242ac120006"
    }
}
data:{
    "code": 0,
    "data": {
        "answer": "I am an intelligent assistant designed to help answer questions by summarizing content from a knowledge base ##0$$. My
        "reference": {
            "total": 1,
            "chunks": [
                {
```

```
                    "id": "faf26c791128f2d5e821f822671063bd",
                    "content": "xxxxxxxx",
                    "document_id": "dd58f58e888511ef89c90242ac120006",
                    "document_name": "1.txt",
                    "dataset_id": "8e83e57a884611ef9d760242ac120006",
                    "image_id": "",
                    "similarity": 0.7,
                    "vector_similarity": 0.0,
                    "term_similarity": 1.0,
                    "positions": [
                        ""
                    ]
                }
            ],
            "doc_aggs": [
                {
                    "doc_name": "1.txt",
                    "doc_id": "dd58f58e888511ef89c90242ac120006",
                    "count": 1
                }
            ]
        },
        "prompt": "xxxxxxxxxxx",
        "id": "a84c5dd4-97b4-4624-8c3b-974012c8000d",
        "session_id": "82b0ab2a9c1911ef9d870242ac120006"
    }
}
data:{
    "code": 0,
    "data": true
}
```


Failure:

```
{
    "code": 102,
    "message": "Please input your question."
}
```

## 与Agent创建会话

**POST** `/api/v1/agents/{agent_id}/sessions`

与Agent创建会话.

### Request

- Method: POST
- URL: `/api/v1/agents/{agent_id}/sessions`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - the required parameters: `str`
  - the optional parameters: `str`
    - `"user_id"`： `string`
      The optional user-defined ID.

#### Request example

If `begin` component in the agent doesn't have required parameters:

```
curl --request POST \
     --url http://{address}/api/v1/agents/{agent_id}/sessions \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '{
     }'
```

If `begin` component in the agent has required parameters:

```
curl --request POST \
     --url http://{address}/api/v1/agents/{agent_id}/sessions \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data '{
            "lang":"Japanese",
            "file":"Who are you"
     }'
```

#### Request parameters

- `agent_id` : (*Path parameter*)
  关联Agent的ID。

### Response

Success:

```
{
    "code": 0,
    "data": {
        "agent_id": "b4a39922b76611efaa1a0242ac120006",
        "dsl": {
```

```
"answer": [],
"components": {
    "Answer:GreenReadersDrum": {
        "downstream": [],
        "obj": {
            "component_name": "Answer",
            "inputs": [],
            "output": null,
            "params": {}
        },
        "upstream": []
    },
    "begin": {
        "downstream": [],
        "obj": {
            "component_name": "Begin",
            "inputs": [],
            "output": {},
            "params": {}
        },
        "upstream": []
    }
},
"embed_id": "",
"graph": {
    "edges": [],
    "nodes": [
        {
            "data": {
                "label": "Begin",
                "name": "begin"
            },
            "dragging": false,
            "height": 44,
            "id": "begin",
            "position": {
```

```
            "x": 53.25688640427177,
            "y": 198.37155679786412
        },
        "positionAbsolute": {
            "x": 53.25688640427177,
            "y": 198.37155679786412
        },
        "selected": false,
        "sourcePosition": "left",
        "targetPosition": "right",
        "type": "beginNode",
        "width": 200
    },
    {
        "data": {
            "form": {},
            "label": "Answer",
            "name": "dialog_0"
        },
        "dragging": false,
        "height": 44,
        "id": "Answer:GreenReadersDrum",
        "position": {
            "x": 360.43473114516974,
            "y": 207.29298425089348
        },
        "positionAbsolute": {
            "x": 360.43473114516974,
            "y": 207.29298425089348
        },
        "selected": false,
        "sourcePosition": "right",
        "targetPosition": "left",
        "type": "logicNode",
        "width": 200
    }
```

```
                ]
            },
            "history": [],
            "messages": [],
            "path": [
                [
                    "begin"
                ],
                []
            ],
            "reference": []
        },
        "id": "2581031eb7a311efb5200242ac120005",
        "message": [
            {
                "content": "Hi! I'm your smart assistant. What can I do for you?",
                "role": "assistant"
            }
        ],
        "source": "agent",
        "user_id": "69736c5e723611efb51b0242ac120007"
    }
}
```


Failure:


```
{
    "code": 102,
    "message": "Agent not found."
}
```


## 与Agent交谈

**POST** `/api/v1/agents/{agent_id}/completions`

向指定的Agent提出一个问题，以启动基于AI的对话。

:::tip NOTE

- 在流模式下，并非所有响应都包含引用，因为这取决于系统的判断。
- 在流模式下，最后一条消息是空消息：

```
data:
{
  "code": 0,
  "data": true
}
```

:::

**Request**

- Method: POST
- URL: `/api/v1/agents/{agent_id}/completions`
- Headers:
  - `'content-Type: application/json'`
  - `'Authorization: Bearer <YOUR_API_KEY>'`
- Body:
  - `"question"：string`
  - `"stream"：boolean`
  - `"session_id"：string`
  - `"user_id"：string` (optional)
  - other parameters: `string`

#### Request example

If the `begin` component doesn't have parameters, the following code will create a session.

```
curl --request POST \
     --url http://{address}/api/v1/agents/{agent_id}/completions \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data-binary '
     {
     }'
```

If the `begin` component have parameters, the following code will create a session.

```
curl --request POST \
     --url http://{address}/api/v1/agents/{agent_id}/completions \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data-binary '
     {
          "lang":"English",
          "file":"How is the weather tomorrow?"
     }'
```

The following code will execute the completion process

```
curl --request POST \
     --url http://{address}/api/v1/agents/{agent_id}/completions \
     --header 'Content-Type: application/json' \
     --header 'Authorization: Bearer <YOUR_API_KEY>' \
     --data-binary '
     {
          "question": "Hello",
```

```
        "stream": true,
        "session_id": "cb2f385cb86211efa36e0242ac120005"
    }'
```

#### Request Parameters

- `agent_id` : (*Path parameter*), `string`
  关联Agent的ID。
- `"question"` : (*Body Parameter*), `string` , *Required*
  开始人工智能对话的问题。
- `"stream"` : (*Body Parameter*), `boolean`
  指示是否以流式方式输出响应：
  - `true` : 启用流（默认）。
  - `false` : 不启用流.
- `"session_id"` : (*Body Parameter*)
  会话的ID。如果没有提供，将生成一个新的会话。
- `"user_id"` : (*Body parameter*), `string`
  可选的用户定义ID。仅在未提供 `session_ID` 时有效。
- Other parameters: (*Body Parameter*)
  开始组件中的参数。

### Response

success without `session_id` provided and with no parameters in the `begin` component:

```
data:{
    "code": 0,
    "message": "",
    "data": {
        "answer": "Hi! I'm your smart assistant. What can I do for you?",
```

```
        "reference": {},
        "id": "31e6091d-88d4-441b-ac65-eae1c055be7b",
        "session_id": "2987ad3eb85f11efb2a70242ac120005"
    }
}
data:{
    "code": 0,
    "message": "",
    "data": true
}
```


Success without `session_id` provided and with parameters in the `begin` component:


```
data:{
    "code": 0,
    "message": "",
    "data": {
        "session_id": "eacb36a0bdff11ef97120242ac120006",
        "answer": "",
        "reference": [],
        "param": [
            {
                "key": "lang",
                "name": "Target Language",
                "optional": false,
                "type": "line",
                "value": "English"
            },
            {
                "key": "file",
                "name": "Files",
                "optional": false,
                "type": "file",
                "value": "How is the weather tomorrow?"
            },
```

```
            {
                "key": "hhyt",
                "name": "hhty",
                "optional": true,
                "type": "line"
            }
        ]
    }
}
data:
```

Success with parameters in the `begin` component:

```
data:{
    "code": 0,
    "message": "",
    "data": {
        "answer": "How",
        "reference": {},
        "id": "0379ac4c-b26b-4a44-8b77-99cebf313fdf",
        "session_id": "4399c7d0b86311efac5b0242ac120005"
    }
}
data:{
    "code": 0,
    "message": "",
    "data": {
        "answer": "How is",
        "reference": {},
        "id": "0379ac4c-b26b-4a44-8b77-99cebf313fdf",
        "session_id": "4399c7d0b86311efac5b0242ac120005"
    }
}
data:{
    "code": 0,
```

```
    "message": "",
    "data": {
        "answer": "How is the",
        "reference": {},
        "id": "0379ac4c-b26b-4a44-8b77-99cebf313fdf",
        "session_id": "4399c7d0b86311efac5b0242ac120005"
    }
}
data:{
    "code": 0,
    "message": "",
    "data": {
        "answer": "How is the weather",
        "reference": {},
        "id": "0379ac4c-b26b-4a44-8b77-99cebf313fdf",
        "session_id": "4399c7d0b86311efac5b0242ac120005"
    }
}
data:{
    "code": 0,
    "message": "",
    "data": {
        "answer": "How is the weather tomorrow",
        "reference": {},
        "id": "0379ac4c-b26b-4a44-8b77-99cebf313fdf",
        "session_id": "4399c7d0b86311efac5b0242ac120005"
    }
}
data:{
    "code": 0,
    "message": "",
    "data": {
        "answer": "How is the weather tomorrow?",
        "reference": {},
        "id": "0379ac4c-b26b-4a44-8b77-99cebf313fdf",
        "session_id": "4399c7d0b86311efac5b0242ac120005"
```

```
    }
}
data:{
    "code": 0,
    "message": "",
    "data": {
        "answer": "How is the weather tomorrow?",
        "reference": {},
        "id": "0379ac4c-b26b-4a44-8b77-99cebf313fdf",
        "session_id": "4399c7d0b86311efac5b0242ac120005"
    }
}
data:{
    "code": 0,
    "message": "",
    "data": true
}
```

Failure:

```
{
    "code": 102,
    "message": "`question` is required."
}
```

## 列出Agent会话

**GET** `/api/v1/agents/{agent_id}/sessions?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&id={session_id}&user_id={user_id}`

列出与指定Agent关联的会话。

### Request

- Method: GET
- URL: `/api/v1/agents/{agent_id}/sessions?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&id={session_id}`
- Headers:
  - `'Authorization: Bearer <YOUR_API_KEY>'`

#### Request example

```
curl --request GET \
     --url http://{address}/api/v1/agents/{agent_id}/sessions?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&id={session_
     --header 'Authorization: Bearer <YOUR_API_KEY>'
```

#### Request Parameters

- `agent_id` : (*Path parameter*)
  关联Agent的ID。
- `page` : (*Filter parameter*), `integer`
  指定显示会话的页面。 默认 `1` .
- `page_size` : (*Filter parameter*), `integer`
  每页上的会话数。 默认 `30` .
- `orderby` : (*Filter parameter*), `string`
  会话排序的字段。可用选项:
  - `create_time` (default)
  - `update_time`
- `desc` : (*Filter parameter*), `boolean`
  指示是否应按降序对检索到的会话进行排序。默认 `true` .
- `id` : (*Filter parameter*), `string`
  要检索的Agent会话的ID。

- `user_id` : (*Filter parameter*), `string`
  创建会话时传入的可选用户定义ID。

### Response

Success:


```
{
    "code": 0,
    "data": {
        "agent_id": "e9e2b9c2b2f911ef801d0242ac120006",
        "dsl": {
            "answer": [],
            "components": {
                "Answer:OrangeTermsBurn": {
                    "downstream": [],
                    "obj": {
                        "component_name": "Answer",
                        "params": {}
                    },
                    "upstream": []
                },
                "Generate:SocialYearsRemain": {
                    "downstream": [],
                    "obj": {
                        "component_name": "Generate",
                        "params": {
                            "cite": true,
                            "frequency_penalty": 0.7,
                            "llm_id": "gpt-4o___OpenAI-API@OpenAI-API-Compatible",
                            "max_tokens": 256,
                            "message_history_window_size": 12,
                            "parameters": [],
                            "presence_penalty": 0.4,
                            "prompt": "Please summarize the following paragraph. Pay attention to the numbers and do not make things up. T
```

```
                "temperature": 0.1,
                "top_p": 0.3
            }
        },
        "upstream": []
    },
    "begin": {
        "downstream": [],
        "obj": {
            "component_name": "Begin",
            "params": {}
        },
        "upstream": []
    }
},
"graph": {
    "edges": [],
    "nodes": [
        {
            "data": {
                "label": "Begin",
                "name": "begin"
            },
            "height": 44,
            "id": "begin",
            "position": {
                "x": 50,
                "y": 200
            },
            "sourcePosition": "left",
            "targetPosition": "right",
            "type": "beginNode",
            "width": 200
        },
        {
            "data": {
```

```
        "form": {
            "cite": true,
            "frequencyPenaltyEnabled": true,
            "frequency_penalty": 0.7,
            "llm_id": "gpt-4o___OpenAI-API@OpenAI-API-Compatible",
            "maxTokensEnabled": true,
            "max_tokens": 256,
            "message_history_window_size": 12,
            "parameters": [],
            "presencePenaltyEnabled": true,
            "presence_penalty": 0.4,
            "prompt": "Please summarize the following paragraph. Pay attention to the numbers and do not make things u
            "temperature": 0.1,
            "temperatureEnabled": true,
            "topPEnabled": true,
            "top_p": 0.3
        },
        "label": "Generate",
        "name": "Generate Answer_0"
    },
    "dragging": false,
    "height": 105,
    "id": "Generate:SocialYearsRemain",
    "position": {
        "x": 561.3457829707513,
        "y": 178.7211182312641
    },
    "positionAbsolute": {
        "x": 561.3457829707513,
        "y": 178.7211182312641
    },
    "selected": true,
    "sourcePosition": "right",
    "targetPosition": "left",
    "type": "generateNode",
    "width": 200
```

```
            },
            {
                "data": {
                    "form": {},
                    "label": "Answer",
                    "name": "Dialogue_0"
                },
                "height": 44,
                "id": "Answer:OrangeTermsBurn",
                "position": {
                    "x": 317.2368194777658,
                    "y": 218.30635555445093
                },
                "sourcePosition": "right",
                "targetPosition": "left",
                "type": "logicNode",
                "width": 200
            }
        ]
    },
    "history": [],
    "messages": [],
    "path": [],
    "reference": []
},
"id": "792dde22b2fa11ef97550242ac120006",
"message": [
    {
        "content": "Hi! I'm your smart assistant. What can I do for you?",
        "role": "assistant"
    }
],
"source": "agent",
"user_id": ""
```

```
    }
}
```

Failure:

```
{
    "code": 102,
    "message": "You don't own the agent ccd2f856b12311ef94ca0242ac1200052."
}
```

# Agent管理

## 列出Agent

**GET** `/api/v1/agents?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&name={agent_name}&id={agent_id}`

列出Agent.

### Request

- Method: GET
- URL: `/api/v1/agents?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&name={agent_name}&id={agent_id}`
- Headers:
  - `'Authorization: Bearer <YOUR_API_KEY>'`

#### Request example

```
curl --request GET \
     --url http://{address}/api/v1/agents?page={page}&page_size={page_size}&orderby={orderby}&desc={desc}&name={agent_name}&id={agent_id}
     --header 'Authorization: Bearer <YOUR_API_KEY>'
```

#### Request parameters

- `page`: (*Filter parameter*), `integer`
  指定显示Agent的页面。默认 `1`.
- `page_size`: (*Filter parameter*), `integer`
  每页上的Agent数量。默认 `30`.
- `orderby`: (*Filter parameter*), `string`
  对结果进行排序的属性。可用选项:
  - `create_time` (default)
  - `update_time`
- `desc`: (*Filter parameter*), `boolean`
  指示是否应按降序对检索到的Agent进行排序。 默认 `true`.
- `id`: (*Filter parameter*), `string`
  要检索的Agent的ID。
- `name`: (*Filter parameter*), `string`
  要检索的Agent的名称。

### Response

Success:

```
{
    "code": 0,
    "data": [
        {
```

```
"avatar": null,
"canvas_type": null,
"create_date": "Thu, 05 Dec 2024 19:10:36 GMT",
"create_time": 1733397036424,
"description": null,
"dsl": {
    "answer": [],
    "components": {
        "begin": {
            "downstream": [],
            "obj": {
                "component_name": "Begin",
                "params": {}
            },
            "upstream": []
        }
    },
    "graph": {
        "edges": [],
        "nodes": [
            {
                "data": {
                    "label": "Begin",
                    "name": "begin"
                },
                "height": 44,
                "id": "begin",
                "position": {
                    "x": 50,
                    "y": 200
                },
                "sourcePosition": "left",
                "targetPosition": "right",
                "type": "beginNode",
                "width": 200
            }
```

```
                    ]
                },
                "history": [],
                "messages": [],
                "path": [],
                "reference": []
            },
            "id": "8d9ca0e2b2f911ef9ca20242ac120006",
            "title": "123465",
            "update_date": "Thu, 05 Dec 2024 19:10:56 GMT",
            "update_time": 1733397056801,
            "user_id": "69736c5e723611efb51b0242ac120007"
        }
    ]
}
```


Failure:


```
{
    "code": 102,
    "message": "The agent doesn't exist."
}
```